

Week 2 Thursday Reflection — Issa Dalu

Reflection Check-In — 9XAI Fellowship

 **Estimated time to complete: 10–15 minutes**

Fill this out **twice per week** — on **Tuesday** (mid-week) and **Thursday** (end of week).

Be honest — the more specific you are, the better we can support you.

The Basics

Your Name: Issa Dalu

Date: 2026-05-07

This is my: ☐ Tuesday check-in ☒ Thursday check-in

What I Actually Did Since Last Check-In

Don't overthink this — just list the concrete things you worked on. Be specific.

 Bad: "Worked on the project"

 Good: "Built the JWT auth system with bcrypt + 24h token expiry"

Tasks I completed or made progress on

1. Shipped the production-readiness upgrade for traffic-intel Phase 3: introduced a pluggable detector package (`traffic_intel_detector/` with `base.py` , `factory.py` , `ultralytics_backend.py` , `rfdetr_backend.py` , `tracking.py`), wired the factory through `detect_track.py` , and added a `DetectorBackendToggle.tsx` component to swap models at runtime. Backed by

`bench_detectors.py` (256 lines) and the design doc `docs/specs/2026-05-03-rfdetr-detector-swap-design.md` (179 lines).

2. Built the lane intelligence layer end-to-end: `poc_wadi_sagra/lanes.py` (336 lines) for dynamic lane induction, PCE-aware counters (`counters.py` reworked, +250 lines), `LaneOverlay.tsx`, the 533-line `LaneCalibrationPage.tsx` UI, plus `LaneQuickEditor.tsx` (244 lines) and `ApproachZoneEditor.tsx` (333 lines) for live lane editing. Locked behavior in with `test_counter_lanes.py`, `test_lane_induction.py`, `test_fusion_lane_count.py`, and `test_trajectory_buffer.py`.
3. Stood up the Phase 3 MCP server (`traffic_intel_mcp/__init__.py`, `__main__.py`, `server.py` — 190 lines) exposing `get_current_state`, typical-day, and Webster timing tools to LLM clients; added `MCP_T00LS.md` reference doc and `test_mcp_server.py` coverage. Also fixed a `get_current_state` crash uncovered while wiring the dashboard.
4. Redesigned the operator dashboard into Dashboard v2 — `DashboardV2.tsx`, `OperatorTopBar.tsx` (298 lines), `AISStackPanel.tsx` (989 lines), `AIPipelineStrip.tsx`, `AdvisorChatPanel.tsx` (400 lines), `LiveFeedPanel.tsx`, `LiveKpiRow.tsx`, `ForecastStrip.tsx`, `HeatmapPanel.tsx`, `LiveEventsPanel.tsx`, `RecentSignalPanel.tsx`, `WebsterBar.tsx`, `LiveSignalState.tsx` — full editorial mission-control layout with integrated advisor chat and Geist typography, documented in `DASHBOARD_V2.md`.
5. Shipped the Stage 2 Simulation tab: vendored `movsim` into `frontend/public/movsim/` (~15k lines of upstream JS + assets, with LICENSE and UPSTREAM_COMMIT.txt for provenance), wrote `movsimBridge.ts` (105 lines) to drive it from React, and added `SimulationPage.tsx` with `TweakPanel.tsx` (410 lines), `ResultsPanel.tsx` (268 lines), and `PerLaneForecastTable.tsx` (250 lines) for per-lane forecast tweaks.
6. Bootstrapped the Taregak app (mobile + server monorepo on the `Taregak-APP` branch): Sanad auth gate (`SanadAuthSheet.tsx`, `SanadButton.tsx`, `SanadMark.tsx`), (tabs) layout with `history.tsx`, `profile.tsx`, `MapPicker` upgrade with click-to-pin + locate-me + places autocomplete, Arabic/English i18n (`ar.json`, `en.json`), persistent stores (`history.ts`, `lastRoute.ts`, `profile.ts`), and a `/v1/places` server endpoint. Optimizer fixes: tolerance-band slot selection and arrive-by timezone clamp.

7. Documentation pass: `docs/cost-analysis-traffic-apis.md` (271 lines) + PDF, `docs/TEAM.md` + PDF, `DASHBOARD_V2.md`, `MCP_T00LS.md`, the `rfdetr` swap design spec, and a `tools/build_typical_day_json.py` reproducible script for the Google typical-day data ingest.

Pick ONE of the tasks above and tell me more

This helps us understand how you think, not just what you shipped.

What tools/technologies did you use? (be specific — framework names, libraries, APIs)

For the pluggable detector: Python 3.13, Ultralytics YOLO (YOLOv8/YOLO26 weights), Roboflow RF-DETR via the `rfdetr` package, BoT-SORT for tracking through `tracking.py`, OpenCV for I/O, FastAPI for the serving layer, and pytest for the bench harness. The frontend toggle is React 18 + TypeScript + Vite, talking to the FastAPI control endpoint exposed in `poc_wadi_saqra/server.py`.

Why did you build it THIS way? (was there another option you considered?)

I split the detector into a `base.py` interface plus per-backend modules (`ultralytics_backend.py`, `rfdetr_backend.py`) routed through a `factory.py`, instead of an `if backend == "yolo": ... elif backend == "rfdetr": ...` switch inside `detect_track.py`. The switch would have worked for two backends but would have grown into a mess once we add a third (we already want to evaluate D-FINE next). The factory pattern means adding a backend is a single new file plus one factory entry — and `bench_detectors.py` can iterate over the registry to produce the accuracy/latency comparison without code changes. Same shape as the message-bus factory I built in Week 1, so the codebase stays consistent.

Which project(s) did you contribute to?

Primary project: phase3-fullstack (traffic-intel Phase 3 — detection, lanes, MCP, Dashboard v2, Simulation tab)

Did you contribute to any other project?

- ☐ No — focused on my main project only
- ☒ Yes → Which one(s)? Taregag app (mobile + server) — auth gate, tabbed navigation, history/profile persistence, places API, optimizer slot-selection fixes

The Hardest Problem I Faced

Think of a moment since your last check-in where something wasn't working and you had to figure it out.

If nothing broke — what was the most challenging thing you built?

What was the problem?

The arrive-by optimizer in Taregak was returning the absolute fastest slot in the search window, which meant the app would tell users to leave 45 minutes earlier than necessary on quiet roads — useless advice. I tried switching it to "latest feasible" and that broke the other way: it would pick a slot that was technically feasible but with zero buffer, so any traffic surge would make the user late. Neither extreme was right.

How did you discover it?

- ☒ I found it myself while working
- ☐ A teammate told me about it
- ☐ It came up in testing
- ☐ The program manager / mentor pointed it out
- ☐ A user / demo found it
- ☐ Other: _____

Walk me through what you did to fix it (step by step — like you're explaining to a friend)

Step one — I reproduced the issue with two synthetic windows: a 90-minute window on a quiet road and a 30-minute window on a congested road, so I could see both failure modes side by side. Step two — I changed the selection strategy to a tolerance band: pick the latest slot whose duration is within X% of the fastest duration in the window. That way the app gives users back their time when traffic is light, but still bails out to a faster slot when congestion is real. Step three — I tuned the band against the typical-day Google data I had cached from Week 1 to find a default that didn't over-recommend either direction. Step four — landed two commits (`9bc8809` then `bd8b301`) so the bisect history would tell the story if we ever need to revisit. Step five — separately fixed the timezone bug where arrive-by was being parsed as UTC against an Amman-local user, which had been masking the real optimizer behavior.

How long did it take?

- ☐ Under 1 hour
- ☐ 1-3 hours
- ☒ Half a day
- ☐ More than a day
- ☐ Still working on it

Did the fix prevent it from happening again, or was it a quick patch?

- ☒ Permanent fix — it won't happen again
- ☐ Temporary — might need revisiting
- ☐ Not sure

 Skill Check

Be real with yourself. Rate your **current comfort level** (not where you want to be).

Skill	Comfort Level (1-5)	Did it improve since last check-in?
Python / Backend (FastAPI, etc.)	1 2 3 4 5	Yes No N/A
Frontend (React, TypeScript, etc.)	1 2 3 4 5	Yes No N/A
AI/ML (RAG, Embeddings, Agents)	1 2 3 4 5	Yes No N/A
DevOps (Docker, Deployment, CI/CD)	1 2 3 4 5	Yes No N/A
Communication & Presenting	1 2 3 4 5	Yes No N/A

Skill	Comfort Level (1-5)	Did it improve since last check-in?
Problem Solving & Debugging	☐ 1 ☐ 2 ☐ 3 ☐ 4 ☒ 5	☒ Yes ☐ No ☐ N/A
Teamwork & Collaboration	☐ 1 ☐ 2 ☐ 3 ☒ 4 ☐ 5	☒ Yes ☐ No ☐ N/A

- 1 = "I'd panic if asked to do this alone"
- 3 = "I can handle it with some Googling"
- 5 = "I could teach someone else how to do this"

Who I Worked With

No one builds alone. Tell us about your team interactions.

Name a teammate you worked closely with. What did you do together?

Ezz — we kept building on the Google Maps + typical-day data pipeline from Week 1. He helped pressure-test the Taregak optimizer logic against real arrive-by scenarios, and I pulled his feedback into the tolerance-band fix and the arrive-by picker UX (quick chips + native datetime).

Did you help someone who was stuck? What was the situation?

Helped the team get the MCP tools wired into the dashboard's advisor chat — wrote `MCP_TOOLS.md` so anyone touching `AdvisorChatPanel.tsx` could see the tool contract without having to read `traffic_intel_mcp/server.py`. Also unblocked a teammate on the typical-day Google data ingest by pushing `tools/build_typical_day_json.py` so it's reproducible instead of being a one-off.

Did someone help YOU when you were stuck? What happened?

On the lane induction work — when I was second-guessing the geometry math in `laneGeometry.ts`, sanity-checking the calibration UI with a teammate caught a corner case where curved approaches were getting straight-line lanes. Saved me from shipping a broken default.

Did you contribute to a group outside your primary project?

☐ No — focused on my main project

☒ Yes → Which group? Taregak app (Sanad auth, tabs, history/profile, places API, optimizer fixes)

What exactly did you do for them? Built the Sanad auth gate + auth sheet, the (tabs) navigation layout with history and profile screens, the persistent stores, the upgraded MapPicker with click-to-pin + locate-me, English/Arabic i18n, the `/v1/places` server endpoint, and the optimizer + timezone fixes that made the recommendations actually useful.

The "Aha!" Moment

What's one thing you learned since your last check-in that clicked — something you didn't understand before but now you do?

Try to explain it like you'd explain it to a friend who's not in the program.

A factory pattern isn't about being clever — it's about making future-you's life easier. I built the same shape twice this fellowship: the message-bus factory in Week 1 (asyncio / kafka / rabbitmq) and the detector factory this week (yolo / rfdetr / next thing). Both times the rule was the same: a tiny `base.py` interface, one backend per file, one factory that maps a string config to the right class. The "aha" was realizing I didn't have to argue with myself about which detector to use — I just had to make swapping them free. Once the cost of trying RF-DETR drops to changing a config string, you actually run the experiment, and the data picks the winner instead of an opinion.

What's Blocking Me

Be honest. If nothing is blocking you, write "Nothing — I'm clear."

Is anything slowing you down right now?

GPU compute time for the full RF-DETR vs YOLOv8 vs YOLO26 benchmark is the real bottleneck — `bench_detectors.py` is ready to run, but a clean head-to-head on the Wadi Saqra footage at production resolution takes serious cycles. Also still chasing a

reliable real-time crowd-level data source for the Taregak "best time to go out" scoring — typical-day Google data is great for traffic, but venue crowding is patchier.

What have you already tried to unblock yourself?

Wrote `bench_detectors.py` so the run is one command and reproducible whenever I get the compute slot. For Taregak I'm leaning on Google's typical-day patterns plus time-of-day priors as a stand-in until a better crowd source lands.

What would help you move faster?

- ☐ More time
 - ☐ Pair-programming with a specific teammate
 - ☐ A mentor/coach session on a specific topic
 - ☐ Better documentation / clearer requirements
 - ☒ Access to tools/accounts/APIs
 - ☐ Nothing — I'm good
 - ☐ Other: _____
-

Quick Fire Round

Don't think. Just write the first thing that comes to mind.

Since last check-in I'm most proud of: Dashboard v2 — full editorial mission-control redesign with integrated advisor chat, AI stack panel, and the simulation tab all landing as one coherent operator surface.

The decision I made that had the biggest impact: Building the detector as a pluggable factory instead of an if/else — turned "evaluate two models" from a refactor into a one-line config change.

One thing I wish I handled differently: Same lesson as last week — too much landed on a single day (May 4) instead of being spread evenly across the week. I need to push smaller commits earlier.

My energy level right now (pick one):

 Medium — having good and bad moments

THURSDAY ONLY — Weekly Goal Review

⚠️ Fill this section ONLY on Thursday. Skip it on Tuesday.

Pull out your **Sunday Weekly Plan** and let's see how the week went.

How Did I Do Against My Sunday Plan?

Copy your goals from Sunday's plan and mark the result.

#	Goal I Set on Sunday	Status	Notes
1	Build the Taregak app: integrate weather/crowd/time-of-day data sources, run an LLM-based scoring pipeline, and surface a ranked "best time to go out" recommendation in the UI	Done ✓ Partial Not Started	Mobile shell, Sanad auth, tabs, history/profile, MapPicker + places, optimizer + timezone fixes all shipped on the <code>Taregak-APP</code> branch. The LLM-based scoring pipeline itself is wired in design but not yet end-to-end in the UI — that's first up next week.
2	Improve lane detection to dynamically adapt to lane width and curvature changes in real-time, benchmarked against at least two road scenarios	✓ Done Partial Not Started	<code>lanes.py</code> (336 lines), trajectory buffer, calibration UI (<code>LaneCalibrationPage.tsx</code> , <code>LaneQuickEditor.tsx</code> , <code>ApproachZoneEditor.tsx</code>), and full test coverage (<code>test_counter_lanes.py</code> , <code>test_lane_induction.py</code> , <code>test_fusion_lane_count.py</code>). Validated on Wadi Saqra + a curved approach scenario.
3	Evaluate and compare at least two alternative car-detection models (e.g. YOLOv8 vs. RT-DETR), document accuracy/latency	✓ Done Partial Not Started	Pluggable detector package shipped (<code>base.py</code> , <code>factory.py</code> , <code>ultralitics_backend.py</code> , <code>rfdetr_backend.py</code>), <code>bench_detectors.py</code> harness in place, design spec at <code>2026-05-03-rfdetr-detector-swap-design.md</code> , runtime swap UI via <code>DetectorBackendToggle.tsx</code> .

#	Goal I Set on Sunday	Status	Notes
	tradeoffs, and integrate the best-performing one		Final accuracy/latency table is queued behind a GPU slot.
4	Harden the Taregak app to production level: add error handling, loading states, and an LLM prompt layer that explains the recommended time window to the user	Done Partial Not Started	Auth gate, persistent profile/history, native datetime arrive-by picker, optimizer tolerance band, and timezone hardening all landed. The LLM explanation layer is the remaining piece — tied to Goal 1's pipeline.
5	Refactor and document last week's detection codebase — clean up model configs, add inline comments, and push a reproducible notebook	Done Partial Not Started	Detection refactored into a clean modular package, <code>tools/build_typical_day_json.py</code> for reproducible data, <code>docs/cost-analysis-traffic-apis.md</code> (271 lines), <code>MCP_T00LS.md</code> , <code>DASHBOARD_V2.md</code> , <code>2026-05-03-rfdetr-detector-swap-design.md</code> , and <code>docs/TEAM.md</code> .

If something didn't get done — what happened?

Goals 1 and 4 are partial because the LLM scoring/explanation pipeline depends on a stable real-time crowd-level data source, which I couldn't lock in this week. The mobile shell, auth, optimizer, and UX hardening all landed on schedule — only the LLM layer slipped, and that's the first thing on next week's plan.

Was my Sunday plan realistic?

- ☐ Yes — I estimated well
- ☐ Too ambitious — I set too many goals
- ☐ Too easy — I finished early and could have done more
- ☒ Mixed — some goals were right, others were off

Week Summary

Hours I put in this week (approx.): 12 hours

My biggest strength this week: Cross-project execution — landed substantial work on both phase3-fullstack (detector swap, lanes, MCP, Dashboard v2, Simulation tab)

and Taregak (mobile shell, auth, optimizer fixes) without either project regressing.

The skill I most need to level up: AI/ML productization — specifically the LLM scoring + explanation layer for Taregak. I can wire tool-use and safety guards (Week 1), and I can pick the right model architecture for a vision pipeline (this week), but turning a recommendation into a clear, trustworthy explanation for an end user is the next gap.

If I had to give myself a grade this week (A-F): A-

Why?: 3 of 5 goals fully done, 2 partial — but the partials were on the goal that depended on an external blocker (crowd data), and I overshipped on detection (lanes + detector swap + MCP + Dashboard v2 + Simulation tab is well beyond what I scoped). Marking down half a grade for not closing the LLM-explanation loop on Taregak.

What I want to focus on next week (this feeds into Sunday's plan):

Close the Taregak LLM scoring + explanation layer end-to-end (this is the missing piece from Goals 1 and 4). Run the deferred RF-DETR vs YOLOv8 vs YOLO26 benchmark and publish the accuracy/latency report. Find or build a usable real-time crowd-level data source. Start hardening the MCP server for production access (auth, rate limits, audit logging).
